

Trajectory - Averaged Feature Remapping — 팀노광탈 방법 요약

2026 오픈소스 SW · AI 딥러닝 해커톤 / Machine Unlearning

최종 private 0.996465 — 4위 (예선 통과선 상위 6팀)

09 - 05 10:49 시점 10위 0.99085 → 마감 4위 0.996465, +0.0056

1. 문제와 채점식

ViT - B/16(MAE) 기반 100클래스 분류기 **M_o**에서 10개 forget 클래스의 영향력만 제거하고 나머지 90개(retain)의 성능은 유지한다. 재학습(from scratch) 금지.

```
retain_drop = max(ref_acc_r - Acc_r, 0)/100      ref_acc_r = 95.899
forget_gap  = |Acc_f - ref_acc_f|/100           ref_acc_f = 0.0
AUS        = (1 - retain_drop) / (1 + forget_gap) ← classifier 출력에서 측정
RUS_o      = harmonic(1 - CKA_f, CKA_r)         ← classifier 직전 임베딩에서 측정
final      = harmonic(AUS, RUS_o)
```

두 지표의 측정 지점이 다르다는 것이 이 문제의 핵심 구조다. AUS는 head *이후*, CKA는 head *이전*에서 잰다. 우리 해법의 출발점이 여기다.

2. 최종 파이프라인

```
M_o
| ① feature remapping 학습 (4600 step, seed 0)
|   학습 중 100 step마다 가중치 스냅샷 저장
▼
스냅샷 25개 (step 2000~4400)
| ② 전부 균등 평균 — 고르지 않는다
▼
평균 가중치
| ③ classifier head만 재보정 (backbone 동결)
▼
최종 모델   Acc_f 0.00 / Acc_r 95.76 / CKA_f 0.0042 / CKA_r 0.9964
            AUS 0.99857 / RUS_o 0.99612 / local 0.99734 → private 0.996465
```

① Feature remapping — 왜 "지우기"가 아니라 "바꿔치기"인가

CKA는 등방 스케일과 직교 변환에 불변이다. 따라서 forget feature를 작게 줄이거나 회전시키는 접근은 **CKA_f** ≈ 1 로 남아 **RUS_o** = 0, 최종 0점이 된다. 샘플들 사이의 2차 구조 자체를 바꿔야 한다.

그래서 매 스텝 forget 이미지마다 retain 배치에서 무작위 파트너를 뽑아 그 파트너의 teacher(M_o) feature를 목표로 삼는다. 파트너가 매번 새로 뽑히므로 구조가 섞여 원본과 decorrelate되고, feature는 분포 안에 머물러 발산하지 않는다.

손실 항	가중치	역할
remap_f	1.0	forget → 무작위 retain 파트너의 teacher feature

손실 항	가중치	역할
<code>cka_f</code>	3.0	미니배치 CKA 직접 최소화 (<code>cka_floor 0.05</code> 로 포화 시 차단)
<code>feat_r / kd_r</code>	2.0 / 2.0	retain feature 코사인 + logit KD
<code>cka_r</code>	8.0	retain 미니배치 CKA 최대화
<code>ce_r / ce_f</code>	0.0	분류 손실은 쓰지 않는다 (③이 담당)

뒤쪽 6블록만 학습하고 final norm은 동결한다(채점되는 feature가 그 직후라 차원별 스케일이 CKA를 직접 왜곡하는 통로가 된다). EMA 0.99.

② 궤적 평균 — 최적점은 마지막 스텝이 아니다

한 학습 궤적 안에서 `CKA_r`은 0.9951~0.9958로 안정적인데 `CKA_f`는 0.0032~0.0156으로 5배 요동친다. 마지막 스텝만 저장하면 그 요동의 한 점을 그대로 받는다.

`CKA_f`는 forget 이미지 1,500장으로 채는 추정량이므로 요동의 상당 부분이 표본 노이즈다. 따라서 로컬 점수가 가장 높은 스냅샷을 고르지 않고 구간 전체를 균등 평균한다. 같은 스냅샷 13개로 두 후보를 만들어 둘 다 제출한 대조 실험:

방식	로컬	private	갭
로컬 최고 1개 선택	0.99729	0.99494	-0.0024
전부 균등 평균	0.99723	0.99634	-0.0009

로컬 차이 0.000006에 private 차이 0.0014. 최대값 선택은 곧 표본 노이즈 선택이다.

평균의 안전 범위는 "한 궤적 안"이다. 시드가 다르면 삭제 방향이 달라 평균이 서로를 상쇄한다 — 두 시드의 궤적을 섞으니 `CKA_f`가 0.0044 → 0.2205로 붕괴했다. 계열이 다른 모델끼리도 마찬가지다(0.0046 → 0.1021).

③ Head 재보정 — 측정 지점 분리의 이용

채점 feature는 classifier 이전이므로 head를 바꿔도 `CKA_f/CKA_r`은 변하지 않는다(세 모델에서 소수점 5자리까지 동일함을 실측). 따라서 unlearning이 끝난 모델의 head만 retain split으로 이어서 학습하면 `RUS_o` 비용 0으로 `AUS`만 회복된다. forget 라벨이 타깃에 한 번도 없으므로 그 행의 logit이 구조적으로 눌러 `Acc_f`가 0으로 간다(하이퍼파라미터 18개 조합 전부에서 재현).

이 단계의 진짜 값어치는 +0.0016이 아니라 새 설계 공간이다. `Acc_f`가 사후에 공짜로 0이 되므로 학습 목표에서 `ce_f`를 지울 수 있고, 그 용량을 `cka_r`로 옮길 수 있다. 그것이 아래 도약을 만들었다.

	<code>CKA_f</code>	<code>CKA_r</code>	합	raw final	head 보정 후
기존 최고(relfe)	0.0085	0.9843	1.9758	0.9917	0.99330
<code>ce_f=0</code>	0.0236	0.9893	1.9657	0.9840	—
+ <code>cka_r</code> 4.0	0.0046	0.9922	1.9876	0.9886	0.99623

두 CKA가 동시에 좋아진 첫 사례다. raw final(0.9886)만 봤으면 기존 최고 (0.9917)보다 낮아 보여 기각했을 실험이다.

3. 이 대회에서 배운 것

(1) 후보는 raw final이 아니라 $(1 - \text{CKA_f}) + \text{CKA_r}$ 합으로 고른다

조화평균의 두 편미분이 0.496 / 0.504로 사실상 같다. 한쪽을 팔아 다른 쪽을 사는 트레이드는 원리적으로 이득이 0이고, 합 자체를 올려야 한다. 그리고 head 보정이 Acc_f 를 사후에 처리하므로 raw final의 AUS 항은 후보 선별에 정보가 되지 않는다.

(2) 로컬 - private 갭은 점수가 아니라 선택량이 만든다

제출 8건에서 예외가 없었다.

선택 방식	갭
게이트만 (임계값 0개)	-0.0002
단일 모델, 탐색 0	-0.0006
퀘적 스냅샷 전부 평균	-0.0009
스냅샷 중 최고 1개 선택	-0.0024
CKA plateau 구간으로 자름 (임계값 2개)	-0.0022
게이트 + 임계값 조정 (9재료 소프)	-0.0033

데이터에서 뽑은 임계값 하나당 약 -0.002를 낸다. 로컬 최고를 제출하면 안 되고, 규칙으로 집계한 것을 제출해야 한다. 실제로 로컬 0.99516짜리가 private 0.99182, 로컬 0.99302짜리가 private 0.99282였다.

(3) 학습 설정 자체도 이전을 바꾼다

선택 방식을 완전히 동일하게 두고 cka_r 만 8 → 12로 올렸더니 로컬은 두 CKA가 모두 좋아졌는데 (0.99640 → 0.99690, 0.00438 → 0.00366) private은 0.99634 → 0.99488로 나빠졌다. retain 압력을 세게 걸수록 표현이 그 표본에 맞춰진다.

(4) 상대 점수 역산에서 얻는 것은 "필요조건"뿐이다

1위 0.9971869에 대해 $\text{AUS} = 1$ 이어도 $\text{RUS} \geq 0.99439$ 가 필요하고 $\text{CKA_r} \leq 1$ 이므로 $\text{CKA_f} \leq 0.0112$ 가 수학적 필요조건이다 — 이건 유효한 정보다. 반면 "우리 두 모델의 지표를 합치면 상대 점수가 재현된다"는 추론은 레벨셋 위 무한히 많은 조합 중 하나를 짚은 것이라 상대의 방법에 대한 증거가 되지 못한다.

(5) 마감 직전 남는 슬롯은 재추첨에 쓴다

같은 레시피에서 평균 구간 · 간격만 바꿔 세 번 뽑았다.

후보	구간 / 간격	로컬	private
mall	2200~4600 / 200	0.99723	0.99634
d25	2200~4600 / 100	0.99721	0.99639
d25b	2000~4400 / 100	0.99734	0.996465

로컬로는 0.0002 안에서 구분되지 않는데 private은 0.0001씩 벌어진다. 이 두 번의 재추첨이 6위권 → 5위 → 4위를 만들었다.

4. 기각된 시도 (측정 근거와 함께)

시도	결과	기각 사유
NegGrad (forget만 ascent)	0.118	retain 붕괴. 핵심은 "잊게 하면서 안 건드리기"
L2 - SP (M_o로 당기기)	합 - 0.023	$CKA_r + 0.0034$ 를 사고 $CKA_f + 0.0268$ 를 냄. 교환비 8:1
M_o 방향 사후 보간	전 방향 손해	위와 같은 트레이드. CKA_f 가 더 빨리 오른다
전역 GPM (직교 투영)	미실행	forget 활성의 90~94%가 retain 부분공간 안
cka_r 12 / 16	private - 0.0015	로컬만 오르고 이전 안 됨
시드 간 소프	CKA_f 0.2205	삭제 방향 상쇄
$\lambda = 4$ 퀘적 평균	CKA_f 0.0130	개별(0.0042~0.0068)보다 나빠짐. $\lambda = 8$ 과 달리 방향이 덜 정렬
스냅샷 간격 조밀화 (clean ablation)	- 0.00016	무효
채점 지점 앞 LayerNorm 사후 최적화	+ 0.0002	이미 최적점
forget 브랜치 증강	미실행	최종 모델의 forget 일반화 격차가 - 0.0131(음수)로 근거 소멸
retain 대배치 / Fisher 앵커 / gradient 투영	전부 대조군 미달	—

5. 후속 과제 — FOS - 11 (Forget - Orthogonal Subspace Editing at Block 11)

전역 GPM은 기각됐지만 딱 한 층이 예외였다. `block11.mlp.fc2` 입력에서:

측정 (M_o, released split, CLS 토큰)	값
retain 95% 부분공간 차원	186 / 3072
forget 에너지 중 retain 부분공간 밖	65.4%
잔차 90%를 설명하는 방향 수	10
$\text{mean} \ V_f^\top h_f \ ^2 / \text{mean} \ V_f^\top h_r \ ^2$	1160배

retain이 이 10개 방향을 사실상 쓰지 않는다. 그 부분공간에만 저랭크 업데이트 $W' = W_o + B V_f^\top$ 를 걸면 학습 파라미터가 7,680개(전체의 0.009%)다.

1200스텝 screening 결과:

lr	CKA_f	CKA_r
1e - 3	1.0 → 0.2986	0.99641 (무손상)
1e - 2	1.0 → 0.1265	0.98734

retain 쪽 가설은 확인됐다 — cka_r 손실 항 없이 파라미터화만으로 CKA_r 이 0.99641로 유지된다(기존 계열이 cka_r 8.0으로 겨우 얻던 값). 미해결은 forget 쪽 속도다. 필요한 CKA_f 는 0.005인데 도달점이 0.127이고, lr을 올리면 CKA_r 이 함께 깎인다. trade-off를 없앤 것이 아니라 교환비를 크게 개선한 것이다.

남은 자유도: rank k 확대(현재 10), 학습 길이(1200 → 4600+), 다층 확장(b6~b8도 5.5~9.8%는 밖), 스냅샷 저장 시 fold/unfold(현재 B가 `state_dict` 밖이라 꺾적 평균을 못 쓴다).

6. 재현

```
python unlearn_remap.py --config configs/final.yaml # ① 36개 스냅샷
python tools/average_snapshots.py <step 2000~4400의 25개> --out models/averaged.pt # ②
python tools/headfit.py models/averaged.pt --out model.pt --epochs 8 --lr 3e-3 # ③
```

학습은 완전히 결정적이다 — 같은 config · seed로 두 번 실행하면 152개 텐서가 비트 단위로 일치한다. 학습률 스케줄이 없어 4600 스텝에서 끊은 꺾적의 스냅샷은 더 긴 실행의 같은 스텝 값과 동일하다.

상세: [STRATEGY.md](#)(§ 1~46, 판단 근거와 기각 사유), [EXPERIMENTS.md](#)(60여 실험 기록), 제출 패키지
~ /submissions/final/Team_노광탈_final.tar.gz.